The agent loop

What an agent is

An AI agent is a loop around a language model, and this cycle is called the agent loop. A language model on its own is a function: text goes in, text comes out, and nothing else happens. An agent is the program that wraps that function in a cycle so the model can take actions, see what happened, and decide what to do next. Each round, the program sends the conversation so far to the model and reads what comes back.

The conversation is just a list of messages kept by your program. It starts with the instruction and the question from the user. Every round adds to it. The model has no memory of its own between calls, so the program sends the whole list each time. This list is the only place the agent keeps its progress.

One round of the loop

A round has four parts. First, the program sends the current list of messages, along with descriptions of the tools the agent may use. Second, the model replies. Third, the program looks at the reply and decides what kind of reply it is. Fourth, the program acts on that decision.

The reply is one of two kinds. If the reply is a request to use a tool, the program runs that tool, adds the result to the conversation, and goes round again. The model can then read the result and choose its next move: ask for another tool, or write an answer. If the reply is plain text, the loop ends and that text is the final answer.

That is the whole idea. Everything else in an agent is detail about how to do these steps safely. For example, a user might ask for the time in Tokyo plus three hours. In the first round the model asks for a clock tool. The program runs it and adds the result. In the second round the model asks for an addition tool. In the third round, with both results in front of it, the model writes the answer in plain text and the loop ends.

What ends the loop

There are three ways a loop can finish. The happy path is the one above: the model replies with plain text, with no tool request, and the program treats that text as the answer. The second way is a failure the program chooses to report: a step limit is reached, which is covered below. The third is an error that cannot be recovered, such as a network failure that keeps happening or a key that the provider rejects.

The model decides when the work is done, but the program decides when to give up. These are separate jobs, and keeping them separate is what makes an agent dependable.

Why every loop needs a step limit

A confused model can keep asking for tools forever. It might call the same tool with the same arguments again and again because the result did not look the way it expected. It might wander between two tools without getting closer to an answer. Nothing inside the model prevents this, because each round looks like a fresh decision.

So the program counts the rounds and stops with an error after a fixed number, such as five. Choose the limit by thinking about the hardest question you expect. If a normal task needs two or three tool calls, a limit of five or eight leaves room for a retry without allowing a runaway. When the limit is reached, raise a clear error that names the limit, so you can see in the logs that the agent did not finish, instead of silently returning half an answer.

A step limit also protects your wallet. Every round is a paid call to the model, and each round sends a longer conversation than the one before, so a runaway loop gets more expensive with every turn.

When a tool fails

Tools fail like any other code. A web request times out, a file is missing, or the model passes an argument of the wrong type. The program should not crash when this happens. Instead, catch the error and send it back to the model as the tool result, in plain words such as "the file was not found". The model can then correct itself, for instance by trying a different file name, or explain the problem to the user.

Sending errors back as results turns a crash into one more piece of information for the next round. The step limit still applies, so a tool that always fails cannot trap the agent.

Seeing inside the loop

Because the loop is ordinary code, you can watch it. Log each round: the messages sent, the tool the model asked for, the arguments, the result, and the final text. When an agent gives a strange answer, this trace shows you where it went wrong. Often the model was fine and the tool returned something unexpected, or the right tool was never offered.

A good habit is to print one short line per tool call while you are building. It costs nothing and it makes the behaviour of the agent something you can read instead of guess.

Putting it together

An agent loop has a small number of moving parts: a list of messages, a model call, a check for tool requests, a way to run tools and record their results, and a counter that stops the cycle. You can write all of it in a few dozen lines. Larger systems add planning, memory and many tools, but they are built on this same cycle, and when something goes wrong in them, this is the first place to look.